Ejercicio 9 — Una mezcla "optimizada" que deja de comparar

Dificultad: ★★ **Conferencias:** C5 (Cotas mínimas)

Enunciado

> *Una mezcla "optimizada" de dos listas ordenadas de largo n deja de comparar cuando a la primera lista le queda un solo elemento, y lo pone al final. Cuenta cuántas comparaciones hace en el peor caso. Sin probarla, usa la sección 6 de la conferencia 5 para decidir si es correcta, y construye con el adversario la entrada en la que falla.*

1 El algoritmo: una mezcla con parada prematura

La variante intenta ahorrar una comparación: mezcla normalmente mientras a la primera lista le queden al menos dos elementos, y cuando queda uno supone que puede colocarlo al final. La pregunta importante no es solo cuánto trabajo ahorra, sino si esa suposición conserva la especificación de una mezcla ordenada.

Especificación. Precondición: A y B están ordenados de forma creciente y tienen el mismo largo $n \geq 1$. Postcondición: devuelve una lista ordenada que contiene exactamente una copia de cada elemento de A y de B.

```
def mezclar_optimizada(A, B, contador=None):
    """Dejar de comparar en cuanto a A le queda un solo elemento: lo pone al final."""
    R, i, j = [], 0, 0
    n = len(A)
    while (n - i) > 1 and j < len(B):
        if contador is not None:
            contador[0] += 1
        if A[i] <= B[j]:
            R.append(A[i]); i += 1
        else:
            R.append(B[j]); j += 1
    # a A le queda <=1 elemento, o B se vacio
    R.extend(B[j:])
    R.extend(A[i:])        # el ultimo elemento de A (si queda) se pone AL FINAL
    return R
```

2 Correctitud

Como en los ejercicios 7 y 8, separamos correctitud parcial, terminación y correctitud total. Aquí el resultado será distinto: la terminación sí se puede demostrar, pero la correctitud parcial es falsa. La cota mínima de C5 permite detectarlo sin comenzar por una batería de pruebas.

Correctitud parcial: qué tendría que garantizar

Para que la postcondición se cumpla, no basta con que R contenga todos los elementos: también debe quedar ordenada. El código sí conserva el multiconjunto de entrada, porque cada elemento se copia una sola vez, pero cuando el ciclo termina por dejar un elemento de A, coloca todo $B[j:]$

antes de $A[i:]$ sin comparar sus fronteras. Por tanto, queda por demostrar si ese orden es siempre válido; la siguiente cota muestra que no puede serlo.

La cota mínima como detector de errores

C5 §6, el teorema que ya tenemos

Mezclar dos listas ordenadas de largo n requiere, en el peor caso, **exactamente** $2n-1$ comparaciones — ningún algoritmo correcto hace menos.

La variante optimizada hace a lo sumo $2n-2 < 2n-1$ comparaciones en **cualquier** entrada (se demuestra en la sección 3). Si fuera correcta **para toda entrada**, contradiría directamente el teorema. Por lo tanto, **tiene que haber alguna entrada en la que se equivoca**: lo sabemos sin ejecutar el algoritmo, comparando únicamente $2n-2$ contra $2n-1$.

Este es exactamente el uso de una cota mínima como **detector de errores** que describe C5 §8: una cota inferior no solo mide rendimiento, también restringe lo que puede hacer cualquier algoritmo correcto. Si un algoritmo de comparaciones resuelve siempre el problema pero usa menos comparaciones que la cota en todas las entradas, hay una contradicción.

Terminación

Sean i y j las cantidades de elementos ya consumidos de A y B. Mientras el ciclo está activo se cumple $i \le n-2$ y $j \le n-1$. Usamos el potencial

$$\Phi = (n-1-i) + (n-j) = 2n-1-(i+j).$$

Es un entero no negativo; cada vuelta consume exactamente un elemento, de A o de B, y por tanto reduce Φ en 1. El ciclo termina cuando a A le queda un solo elemento o cuando B se vacía. Las operaciones `extend` posteriores recorren listas finitas, así que el procedimiento completo termina.

Correctitud total

La terminación está garantizada, pero la postcondición no: la construcción adversaria de la sección 4 produce una salida que contiene todos los elementos, aunque no está ordenada. Por eso la versión optimizada no tiene correctitud total. La cota mínima demuestra que el fallo debe existir; el adversario muestra exactamente dónde aparece.

3 Costo: cuántas comparaciones hace

Cada vuelta del ciclo hace una comparación y consume un elemento. Si el ciclo termina porque a A le queda uno, se han consumido como máximo $n-1$ elementos de A y, como $j < n$, como máximo $n-1$ de B. Si termina porque B se vacía, entonces se han consumido n elementos de B y, antes de la última comparación, quedaban al menos dos elementos de A, de modo que se consumieron como máximo $n-2$ de A. En ambos casos, el número de comparaciones es como máximo

$$(n-1) + (n-1) = 2n-2.$$

La mezcla correcta también tarda $O(n)$ tiempo, pero su peor caso necesita $2n-1$ comparaciones. La variante ahorra una comparación como máximo, a cambio de perder la garantía de orden. La salida ocupa $O(n)$ espacio; si no contamos el arreglo de salida, el algoritmo con índices necesita espacio auxiliar $O(1)$ (el listado de Python copia la cola con las rebanadas `B[j:]` y `A[i:]`, que añade $O(n)$ temporal).

Verificado con un barrido aleatorio de 300 instancias por tamaño:

```
   n  max comp. optimizada 2n-2 max comp. correcta 2n-1
   2                    2     2                  3     3
   5                    8     8                  9     9
  10                   18    18                 19    19
  50                   98    98                 99    99
 200                  398   398                399   399
1000                 1998  1998               1999  1999
```

En cada tamaño medido la variante alcanza $2n-2$ (la columna "max comp. optimizada" coincide con la de $2n-2$), así que la cota no es floja, y la mezcla correcta, medida con el mismo barrido, alcanza $2n-1$, como dice C5 §6.

4 Construcción del adversario: la entrada en la que falla

C5 §6 construye la cota $2n-1$ con la entrada **intercalada perfecta** $a_1 < b_1 < a_2 < b_2 < \cdots < a_n < b_n$ — la reutilizamos íntegra, sin cambiar nada, porque es exactamente la entrada diseñada para que un algoritmo de mezcla tenga que comparar *todos* los pares de vecinos.

```
n=4: A=[1, 3, 5, 7] B=[2, 4, 6, 8]
     salida optimizada: [1, 2, 3, 4, 5, 6, 8, 7]
     correcta (ordenada)?: False (deberia ser: False)
n=5: A=[1, 3, 5, 7, 9] B=[2, 4, 6, 8, 10]
     salida optimizada: [1, 2, 3, 4, 5, 6, 7, 8, 10, 9]
     correcta (ordenada)?: False (deberia ser: False)
n=8: A=[1, 3, 5, 7, 9, 11, 13, 15] B=[2, 4, 6, 8, 10, 12, 14, 16]
     salida optimizada: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 16, 15]
     correcta (ordenada)?: False (deberia ser: False)
```

Por qué falla, exactamente. Con esta entrada, la mezcla normal va alternando $a_1, b_1, a_2, b_2, \ldots$ Justo cuando a A le queda un solo elemento (a_n), ya se consumieron $a_1, \ldots, a_{n-1}$ y $b_1, \ldots, b_{n-1}$ — a B le queda un solo elemento, b_n. El atajo pone ese elemento de B primero y después, al final de todo, el a_n que quedaba pendiente. Pero en la cadena, $b_{n-1} < a_n < b_n$: el orden correcto termina en $\ldots, b_{n-1}, a_n, b_n$, y el atajo produce $\ldots, b_{n-1}, b_n, a_n$ — a_n y b_n quedan invertidos. Se ve exactamente así en la salida de $n = 4$: termina en $\ldots, 6, 8, 7$ en vez de $\ldots, 6, 7, 8$.

5 Generadores dirigidos y datos extra

El contraejemplo intercalado demuestra el fallo, pero una sola entrada no muestra qué ocurre en otras formas de detener el ciclo. Por eso `verif.py` incluye generadores dirigidos: algunos son controles en los que la parada prematura sí es segura y otros producen distintos tipos de salida rota. Además de comprobar si la salida está ordenada, se cuentan sus **inversiones**, es decir, pares de elementos que aparecen en el orden contrario al correcto. Así medimos no solo si falla, sino cuánto se desordena la salida.

Generador	Construcción	Qué comprueba
`gen_B_se_vacia`	B contiene valores menores que todos los de A.	B se consume completamente; la parada ocurre por la condición normal y la salida es correcta.
`gen_parada_correcta`	Quedan r valores de B, todos menores que el último de A.	La variante deja de comparar, pero en este caso concreto colocar B antes del último de A sí es seguro.
`gen_una_inversion`	Queda un valor de B mayor que el último de A.	Produce una única inversión al copiar ese valor antes del último de A.
`gen_varias_inversiones`	Quedan r valores de B, todos mayores que el último de A.	Produce r inversiones; muestra que el error puede crecer con el tamaño de la cola.
`gen_cola_mixta`	La cola de B contiene valores menores y mayores que el último de A.	Separa los valores que deberían ir antes y después, pero el atajo los copia todos antes.
`gen_intercalada`	$A = (1, 3, \ldots, 2n-1)$ y $B = (2, 4, \ldots, 2n)$.	Es el adversario de C5 y fuerza la inversión final clásica.
`gen_aleatoria_ordenada`	Dos listas ordenadas obtenidas con semillas fijas.	Mide la frecuencia del fallo en entradas no diseñadas a mano.

La salida resumida de la batería es la siguiente. “Comp. media” y “comp. máx.” cuentan las comparaciones; “inv. media” y “inv. máx.” cuentan las inversiones de la salida:

```
             generador casos  fallos comp.media comp.max inv.media inv.max
          B se vacia    29   0.00%      16.00       30      0.00       0
      parada correcta   28   0.00%      29.04       56      0.00       0
        una inversion   29 100.00%      29.00       57      1.00       1
    varias inversiones  25 100.00%      30.00       54      5.00       5
           cola mixta   26 100.00%      30.00       55      2.00       2
intercalada adversaria  29 100.00%      29.00       57      1.00       1
  aleatorias ordenadas 1000  51.60%      28.63       58      0.91       7
```

Estos resultados añaden tres matices a la demostración. Primero, no toda parada temprana falla: los dos generadores de control tienen 0 % de fallos. Segundo, cuando falla, la salida puede tener una inversión, varias, o solo las de los valores de la cola que superan al último de A (cola mixta), según la cola que quedó sin comparar. Tercero, en 1000 entradas aleatorias ordenadas el fallo aparece en el 51.60 % de los casos; por tanto, el adversario no es un fenómeno aislado, aunque la prueba de la cota mínima sigue siendo la demostración general.

```
ejemplos de salidas rotas:
  una inversion: A=[0, 2, 4, 6, 8, 10, 12, 16], B=[1, 3, 5, 7, 9, 11, 13, 17]
    salida=[..., 10, 11, 12, 13, 17, 16], comparaciones=13, inversiones=1
  varias inversiones: A=[0, 2, 4, 6, 8, 10, 12, 16], B=[1, 3, 5, 7, 9, 17, 18, 19]
    salida=[..., 10, 12, 17, 18, 19, 16], comparaciones=12, inversiones=3
  cola mixta: A=[0, 2, 4, 6, 8, 10, 12, 19], B=[1, 3, 5, 7, 17, 18, 21, 22]
    salida=[..., 12, 17, 18, 21, 22, 19], comparaciones=11, inversiones=2
  intercalada adversaria: A=[1, 3, 5, 7], B=[2, 4, 6, 8]
    salida=[..., 3, 4, 5, 6, 8, 7], comparaciones=5, inversiones=1
```

Respuesta a la pregunta del enunciado

Conclusión

- **Comparaciones:** la variante hace como máximo $2n-2$ comparaciones, una menos que la cota mínima $2n-1$ de una mezcla correcta en el peor caso.
- **Correctitud:** no es correcta para todas las entradas. La cota de C5 permite saberlo sin probarla, porque un algoritmo correcto no puede resolver siempre la mezcla usando menos que la cota mínima.
- **Adversario:** las listas intercaladas $A = [1, 3, \ldots, 2n-1]$ y $B = [2, 4, \ldots, 2n]$ hacen que el algoritmo termine con $\ldots, 2n-2, 2n, 2n-1$ en lugar de $\ldots, 2n-2, 2n-1, 2n$.
- **Terminación y costo:** el procedimiento sí termina, tarda $O(n)$ y usa $O(1)$ espacio auxiliar con índices, aparte de la lista de salida, pero el ahorro de una comparación destruye la postcondición.

Errores que hay que vigilar

- **Pensar que el error es raro, como los de las conferencias 2 y 5.** Los errores plantados de C2 §7 (cerca del 1 % de las instancias pequeñas y ninguna de las grandes) y de C5 §8 (probabilidad $1/(n-1)$, 0.12 % con $n = 1024$) casi no se ven con entradas aleatorias — hace falta un adversario cuidadoso para encontrarlos. **Este no es de esa clase**: medido sobre 20,000 pares de arreglos aleatorios uniformes (tamaños entre 2 y 30), falla en

  ```
  entradas aleatorias uniformes: 9834/20000 fallan (49.17 %)
  ```

  casi la **mitad** de las veces. La razón es estructural, no un caso extremo: el atajo falla cada vez que, en el momento en que A queda con un solo elemento, a B todavía le queda algún elemento *mayor* que ese último de A (equivale a $b_n > a_n$, es decir, a que el máximo de B sea mayor que el de A) — y con datos genéricos eso pasa casi la mitad de las veces, no es lo raro. La lección de C2 §6 (el 7 "es una garantía y no una descripción": el caso típico no delata el error) también vale al revés: a veces el caso típico *sí* delata el error, y una batería de pruebas al azar basta para sospecharlo.
- **Confundir "hace menos comparaciones" con "es más rápido y sigue siendo correcto".** Reducir el conteo de comparaciones, en el peor caso, por debajo de una cota demostrada para el problema **siempre** significa que el algoritmo ya no resuelve el problema en todas las entradas (o que la cota no aplica porque cambió el modelo — no es el caso aquí, la mezcla sigue siendo un algoritmo de comparaciones puro).
- **Arreglarlo mal, comparando una vez más "por si acaso".** Agregar una comparación extra entre el último elemento de A y el último de B antes de hacer `extend` no alcanza en general: si a B le quedan **varios** elementos mayores que el último de A (no solo el último de B), una sola comparación adicional no alcanza para ubicarlo entre ellos. El arreglo correcto es simplemente no cortar el ciclo principal hasta que uno de los dos arreglos se vacíe del todo — la mezcla estándar de C5 §6, sin atajos.